



life.augmented

Code execution from internal memory Hands On



Execution from internal flash

- Purpose:
 - Execute code from internal flash and compare different performance scenarios
 - Observe and control the firmware with STM32CubeMonitor
 - Stress the internal ICache by adding additional bandwidth
 - Drawing picture on the screen
 - Audio playback
 - Stress the internal cache by increasing the code size
 - Compare internal flash performance with external flash (in later hands-on)
 - Be able to perform benchmarks suitable for your application
- Hands-on scenario:
 - Import project
 - Configure place code to ITCM RAM (instruction tightly coupled memory) and change linkerscript
 - Change different parameters with STM32CubeMonitor and observe performance

Theory

Code benchmark (1/2)

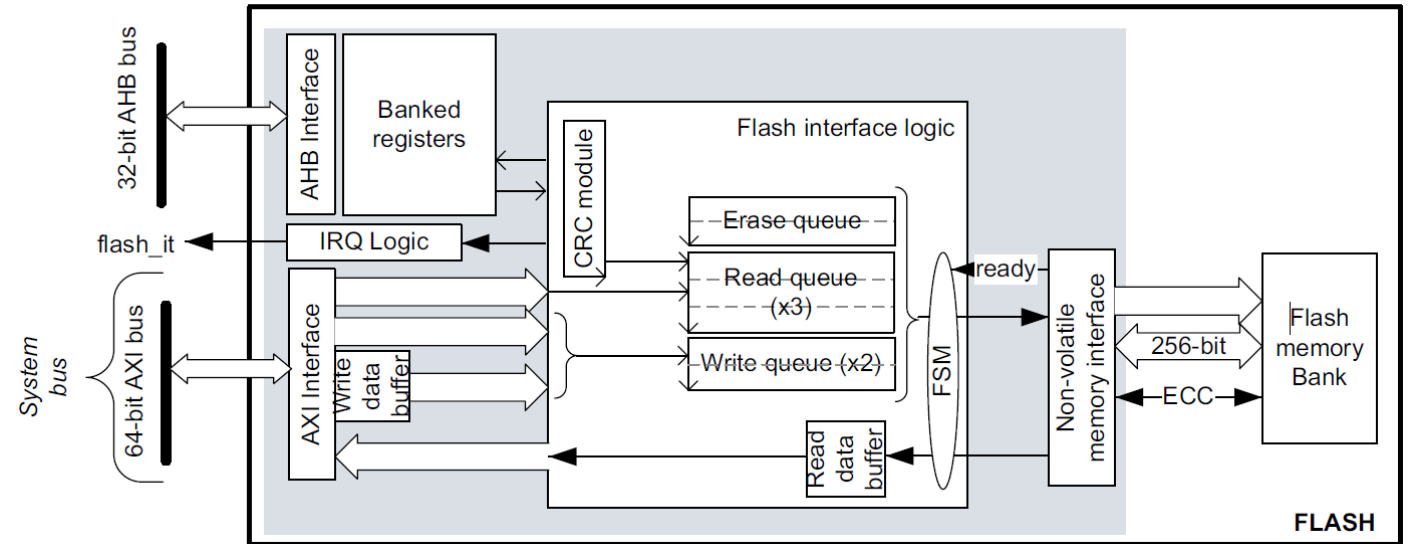
- Executing simple functions multiple times in loop
- Each function is based on same template, but uses different constants
 - Each function has size of 32 bytes (size of single cache line)
 - Only instruction (no data fetch from read-only memory)
- We can change number of functions executed in loop
 - By this we can increase/decrease the size of code that is being cached
- Performance is measured by number of cycles that it takes to execute single function
 - Lower value = better performance

Code benchmark (2/2)

- -O3 flag configured for the benchmark code
- Main loop placed in ITCM RAM
 - To prevent interfering with the measurement of functions
 - We will add this placement in the hands-on
- Exceeding the cache size will affect the performance
 - In this case the effect will be more significant, since all the code is executed sequentially
 - When we reach end of the sequence, the beginning will probably be not kept in cache
 - In “normal” application with some code loops, the effect will be less significant
- Might not be representative to real specific application

Internal flash

- 256-bit flash words
- 3-wait states at maximum clock frequency
 - 275 MHz clock for AXI bus and Flash
- Connected via 64-bit AXI bus
- In sequential read
 - It delivers 64-bit word (8 bytes) on AXI bus each clock cycle (275MHz)
 - Theoretical maximum bandwidth 2.2 GB/s (*)
 - Feeds 32-bit instruction to core each core clock cycle or 2x 16-bit instructions
 - This might not be enough since Cortex-M7 can process two instruction at the same time



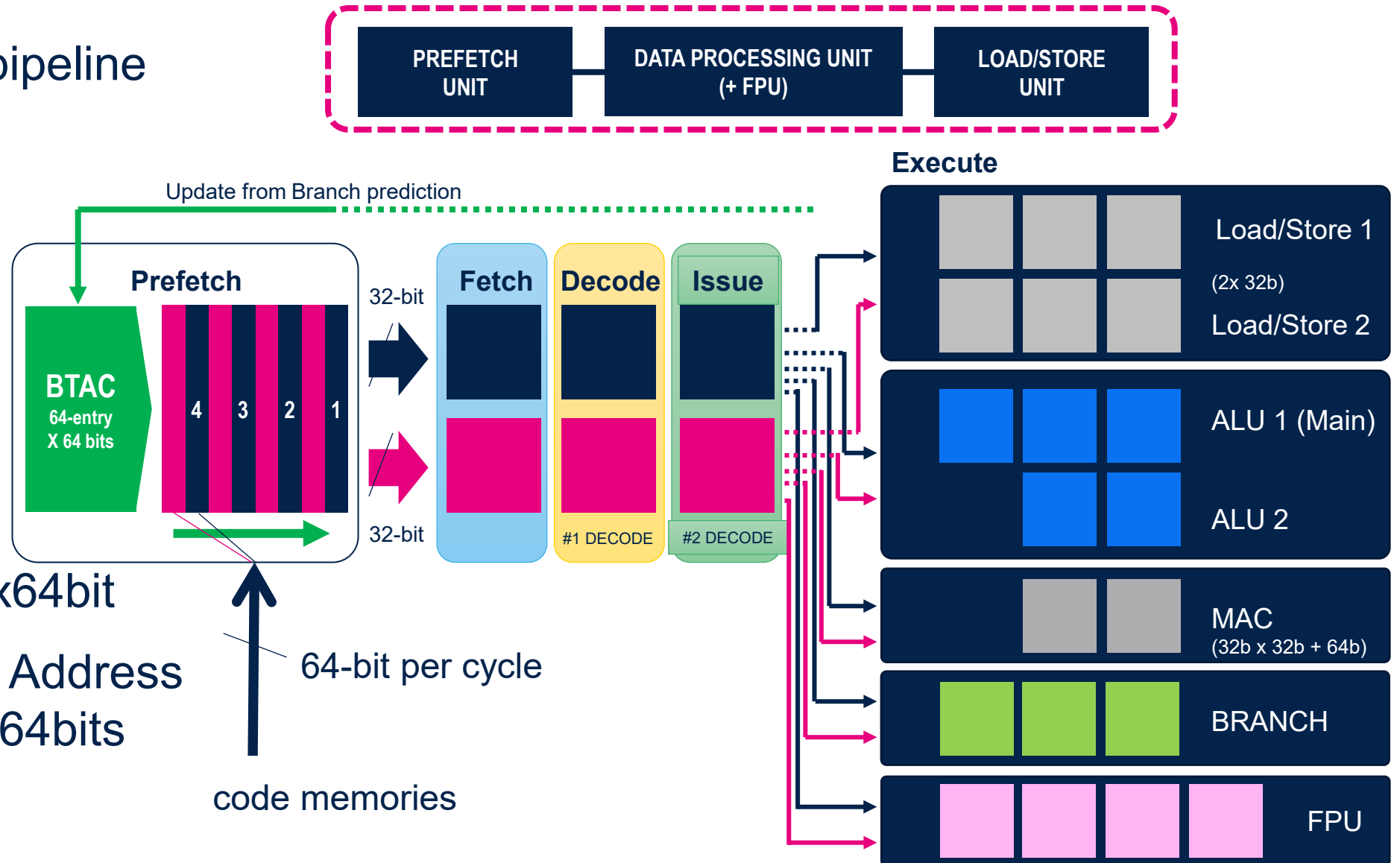
M7 Core pipeline

- M7 six stage pipeline

- Dual issue

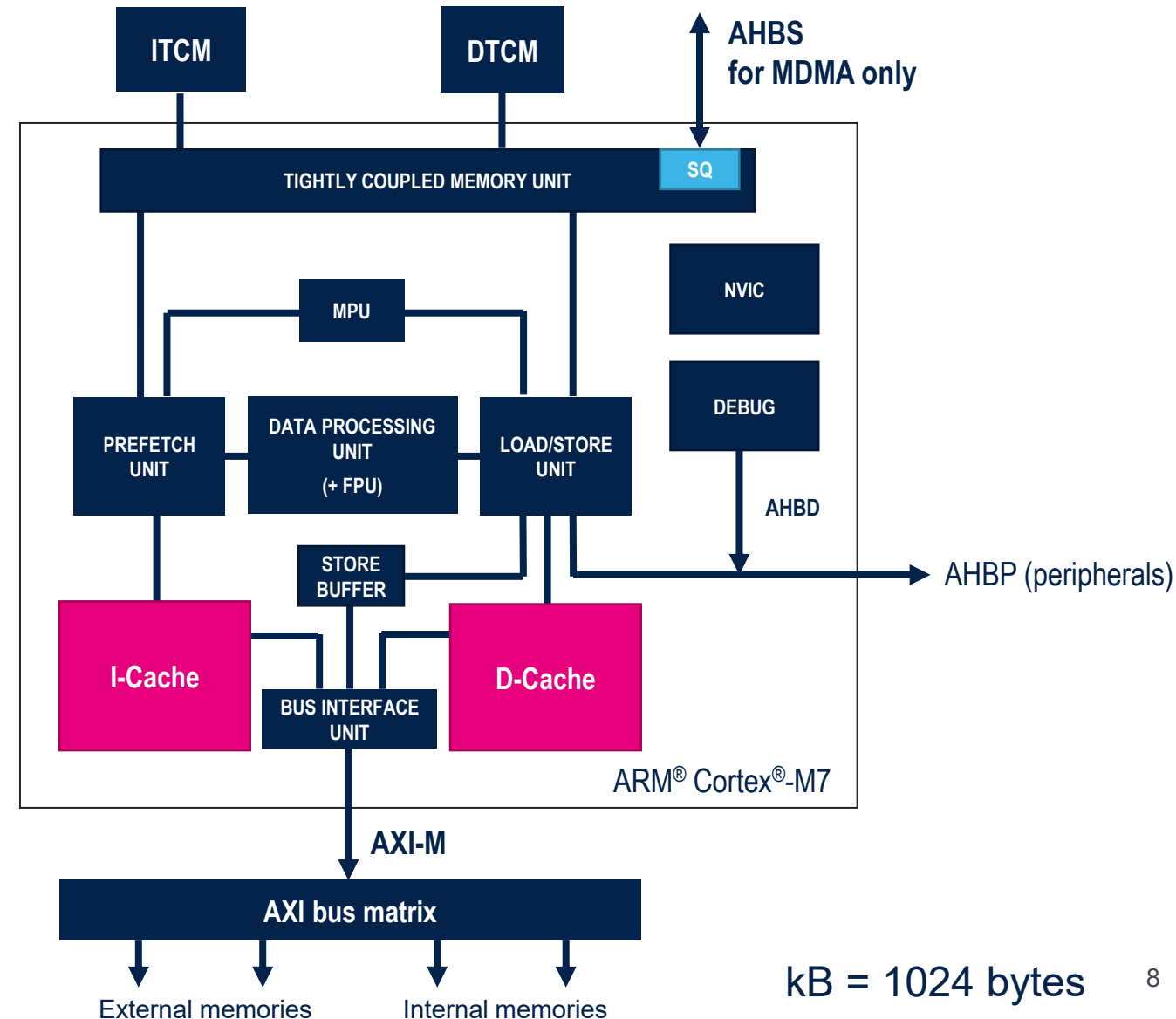
- Loading and executing two instructions in same time

- Prefetch for 4x64bit
- Branch Target Address Cache for 64x64bits



M7 Core Architecture

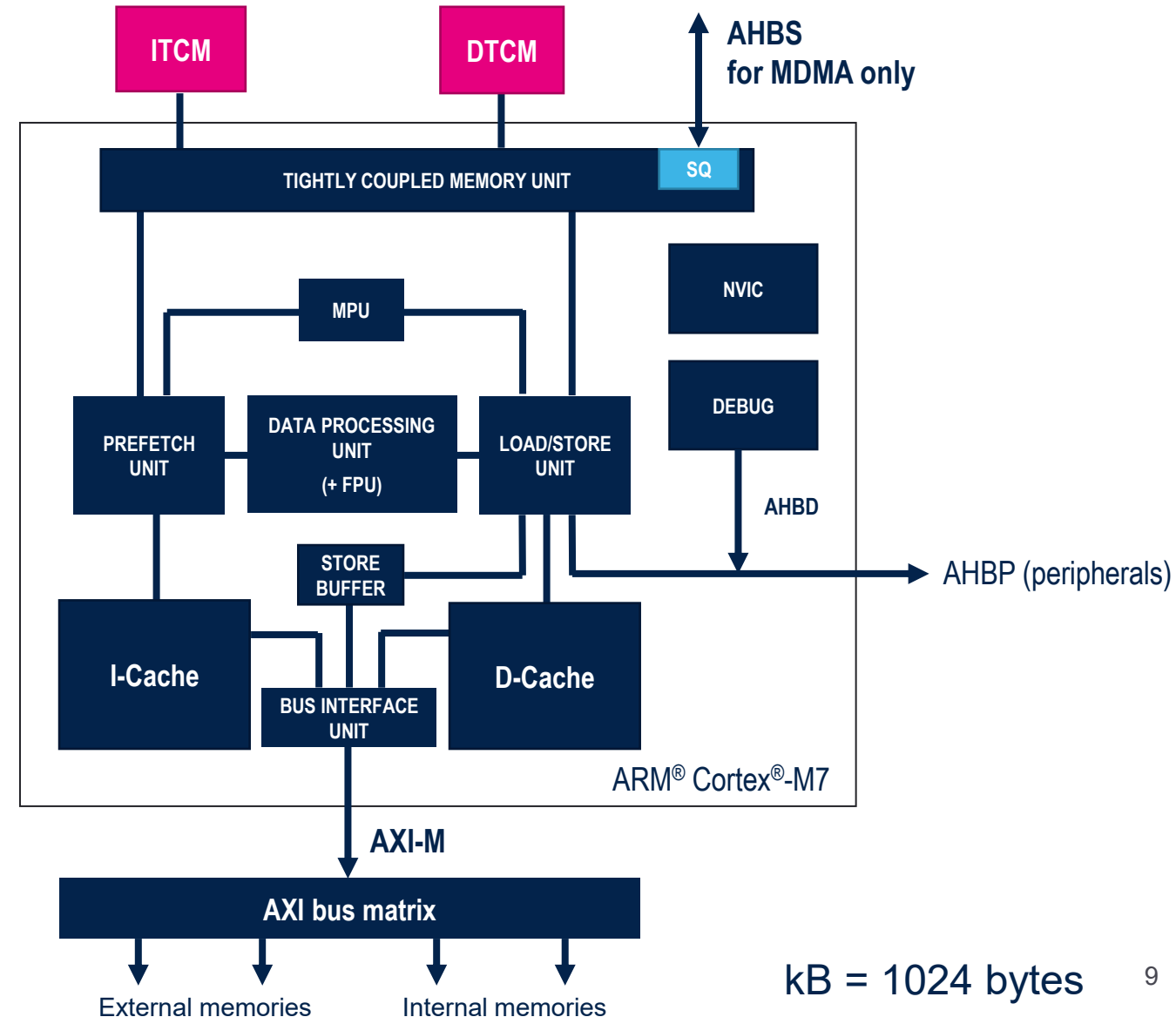
- STM32H72x/73x
 - 32 kB of instruction cache
 - 32 kB of data cache
 - Size depends on the device
- 32 bytes cache line
 - This is the smallest unit of memory stored in cache
- Can be enabled/disabled for parts of memories via MPU



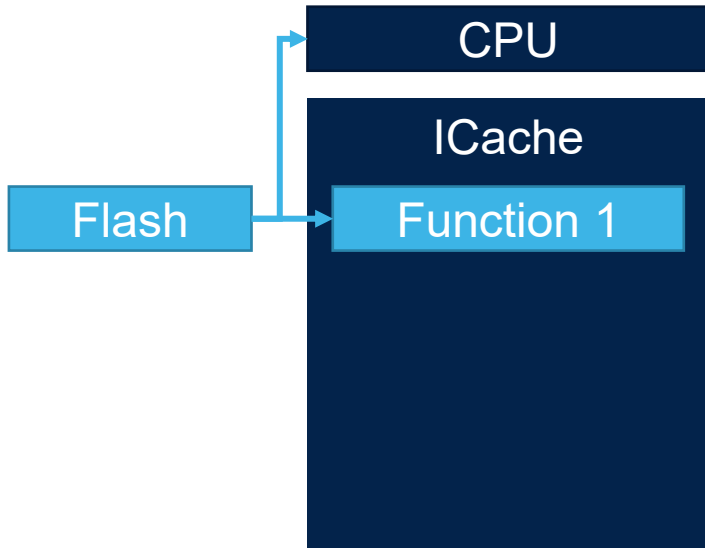
M7 Core Architecture

- Tightly coupled memories for instructions and data (ITCM and DTCM)
 - Useful for placing critical code and data
 - Accessible only by core and MDMA
- Size depends on device and configuration
- DTCM has 128kB
- ITCM has 64kB + additional 192kB

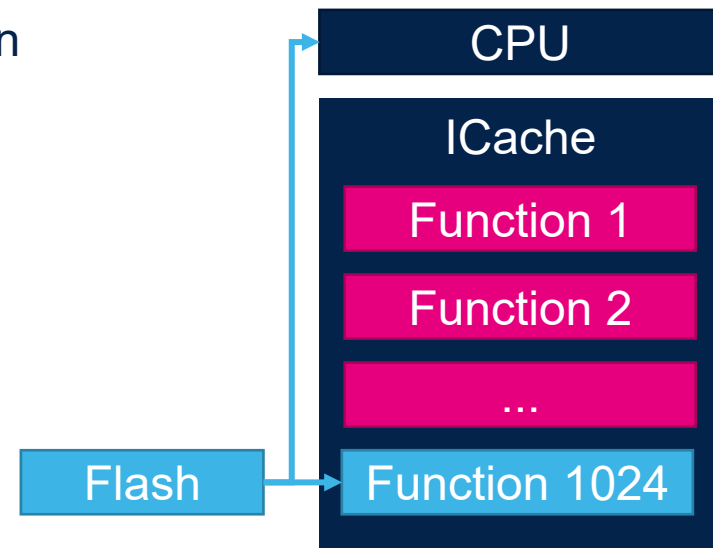
Option	ITCM	AXI SRAM
00	64 kB	320 kB
01	128 kB	256 kB
10	192 kB	192 kB
11	256 kB	128 kB



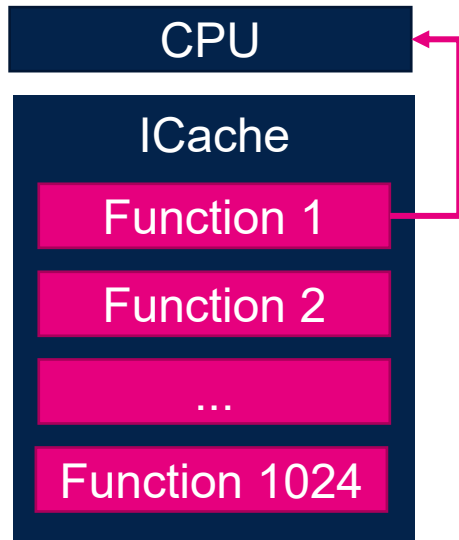
Cache filling – small code



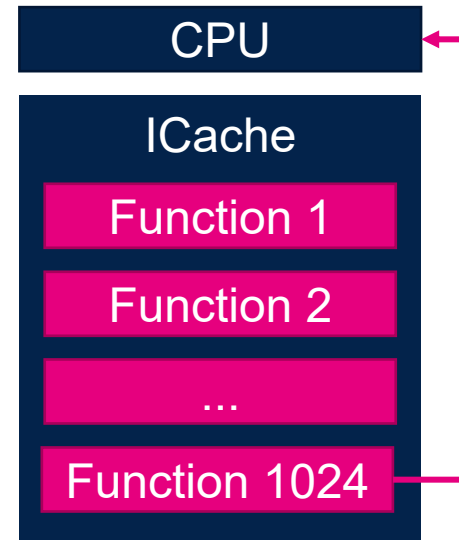
First iteration



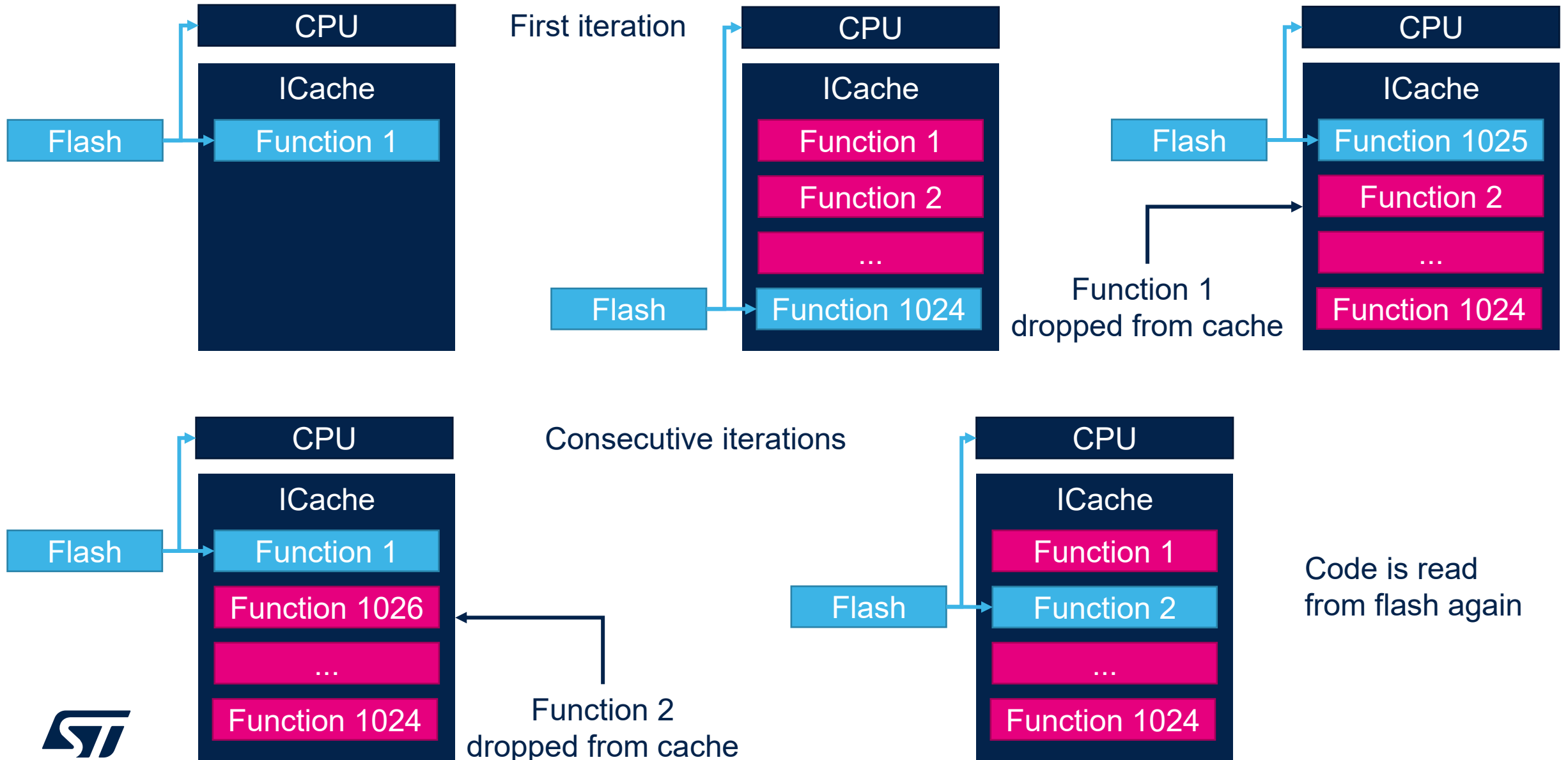
All code loaded to the cache



Consecutive iterations



Cache filling – cache overload



Cache filling – cache overload

- Example on previous slide is a corner case
 - Code fully sequential
 - One big loop
- Cache will behave slightly different
 - There are some alignment restrictions
 - E.g. “Function 1” can be placed only in certain parts of cache

- LTDC refreshes the display periodically
 - In our configuration the rate is 60Hz
- For each refresh it fetch data for active part of display
- The goal is to simulate bandwidth required by graphical applications
- There is no data fetching during blanking period
=> there might be spike changes in performance
- When LTDC disabled, it won't fetch any data
- Parameters: 480×272, 3 Bytes per pixel (BPP), 60Hz
- Required bandwidth: 23.5 MB/s

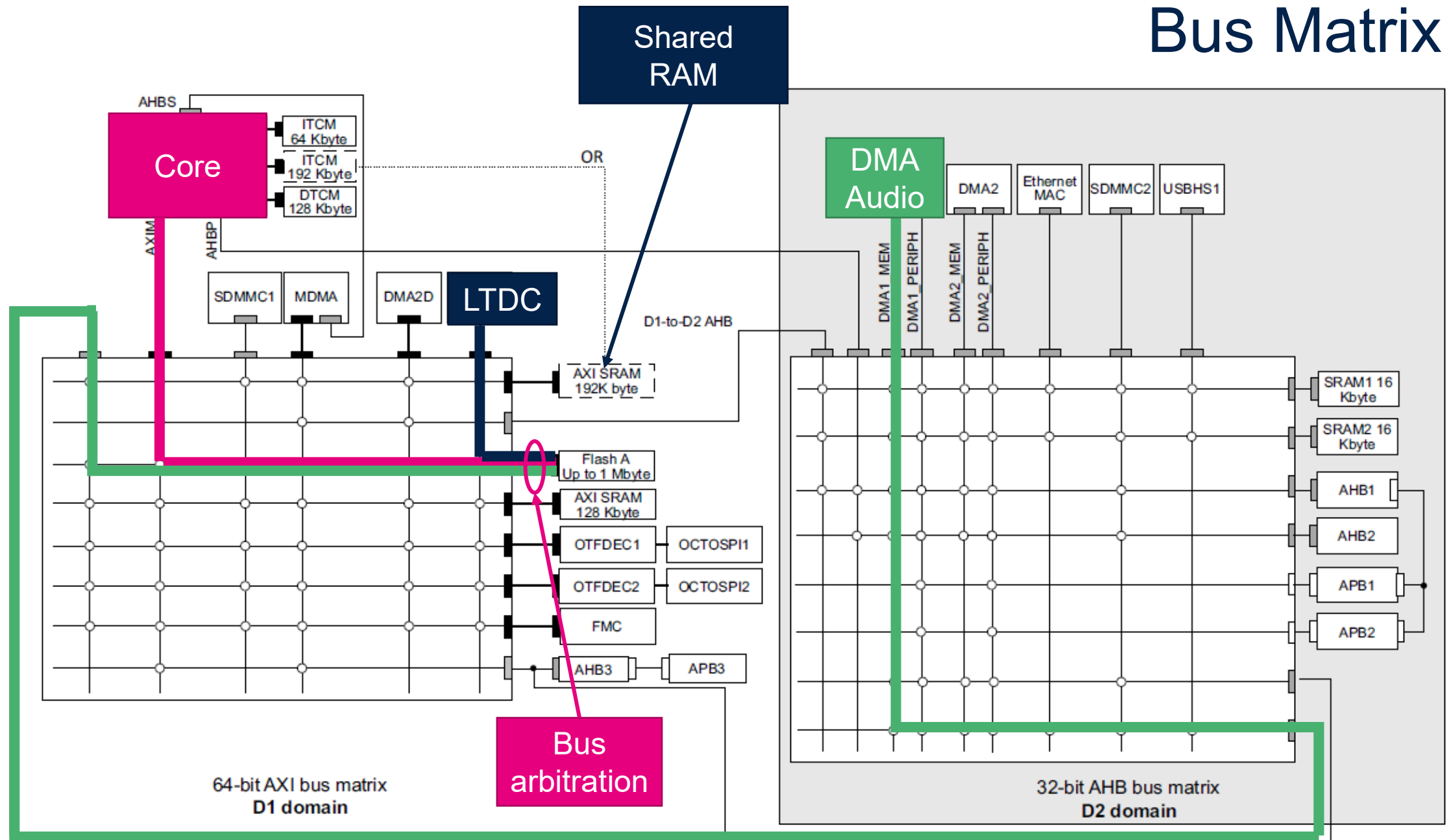


Blanking
(no data fetching)

Audio playback

- Audio samples stored in the FLASH memory
 - Dump of WAV file – WAV header + raw data
- Data fetched by DMA through D2-to-D1 bridge
- Data sent via SAI interface to on-board audio codec
- Sound loop available on “LINE OUT” jack (green)
- Parameters: 16kHz, 16-bit, stereo
- Required bandwidth: 64.0 kB/s

Bus Matrix



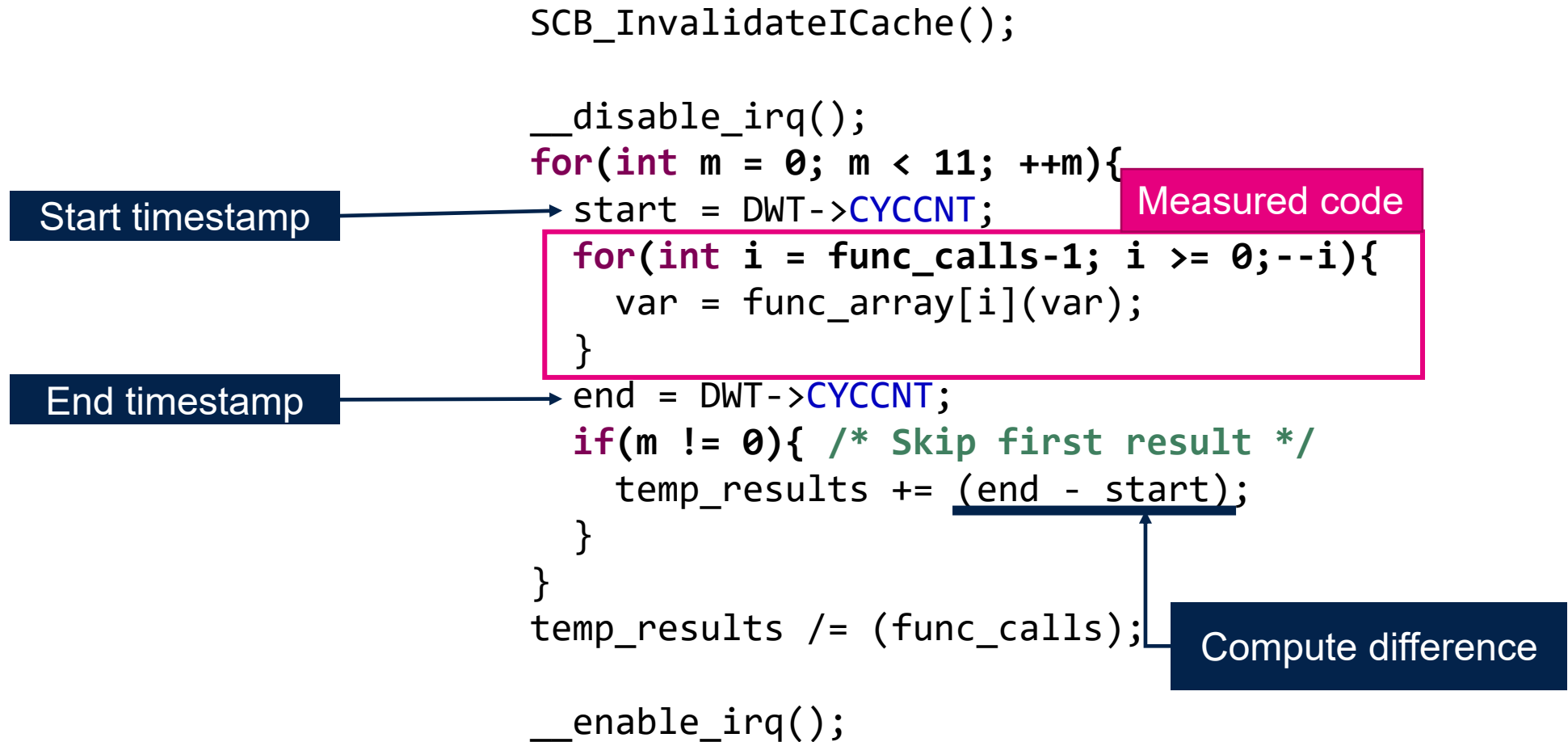
Measuring performance with DWT (1/2)

- DWT (data watchpoint and trace) can be used to measure number of CPU cycles
- DWT contains 32-bit free running counter
- Comparing two timestamps gets us number of CPU cycles
 - @550MHz the counter overflows each ~7.8 seconds
=> for measuring longer intervals other method must be used
 - Lower number => better performance
- Is optional part of the Cortex-M core
 - Available most STM32, except for Cortex-M0/M0+ cores

Measuring performance with DWT (2/2)

- It is always good to make several measurements and make average
 - 1st run of code might be slower due to ICache filling and Branch predictor
 - So it might be better to ignore the first result
- Make sure we measure the functions only
- Disable interrupts

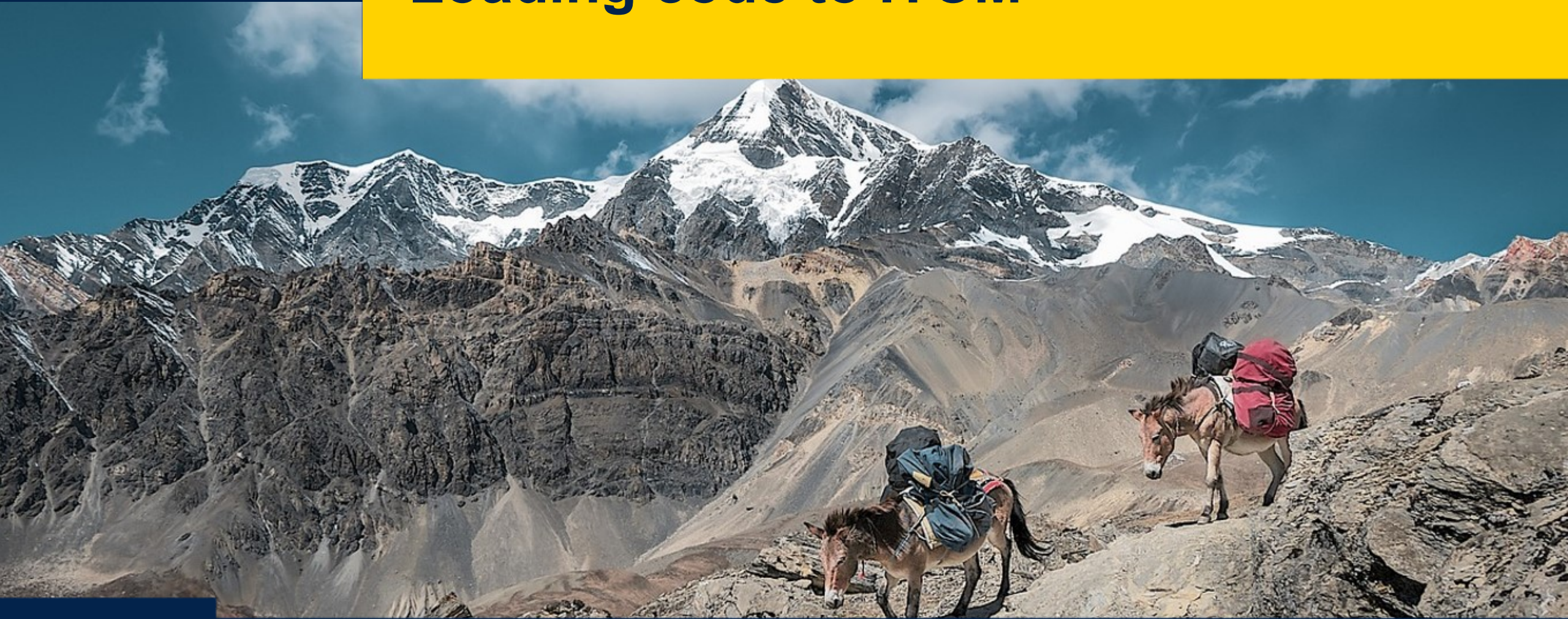
DWT measurement used in Hands On code



Performance tips

- Enable compiler optimization
- Enable caches
 - Data cache might cause some issues with DMA if not handled properly
- Place interrupt routines or critical code into ITCM
 - Fast random access, doesn't "disturb" ICache used by main thread
 - Interrupt routines are usually sequential (no loops), so they don't benefit from ICache that much

Loading code to ITCM



Loading code to ITCM

- Steps:
 - Import prepared project template
 - Place selected function to dedicated section
 - Modify linkerscript
 - Load the data from flash at startup (since ITCM is volatile memory)
 - Check with Build Analyzer
 - Setup debug configuration
 - Run the code



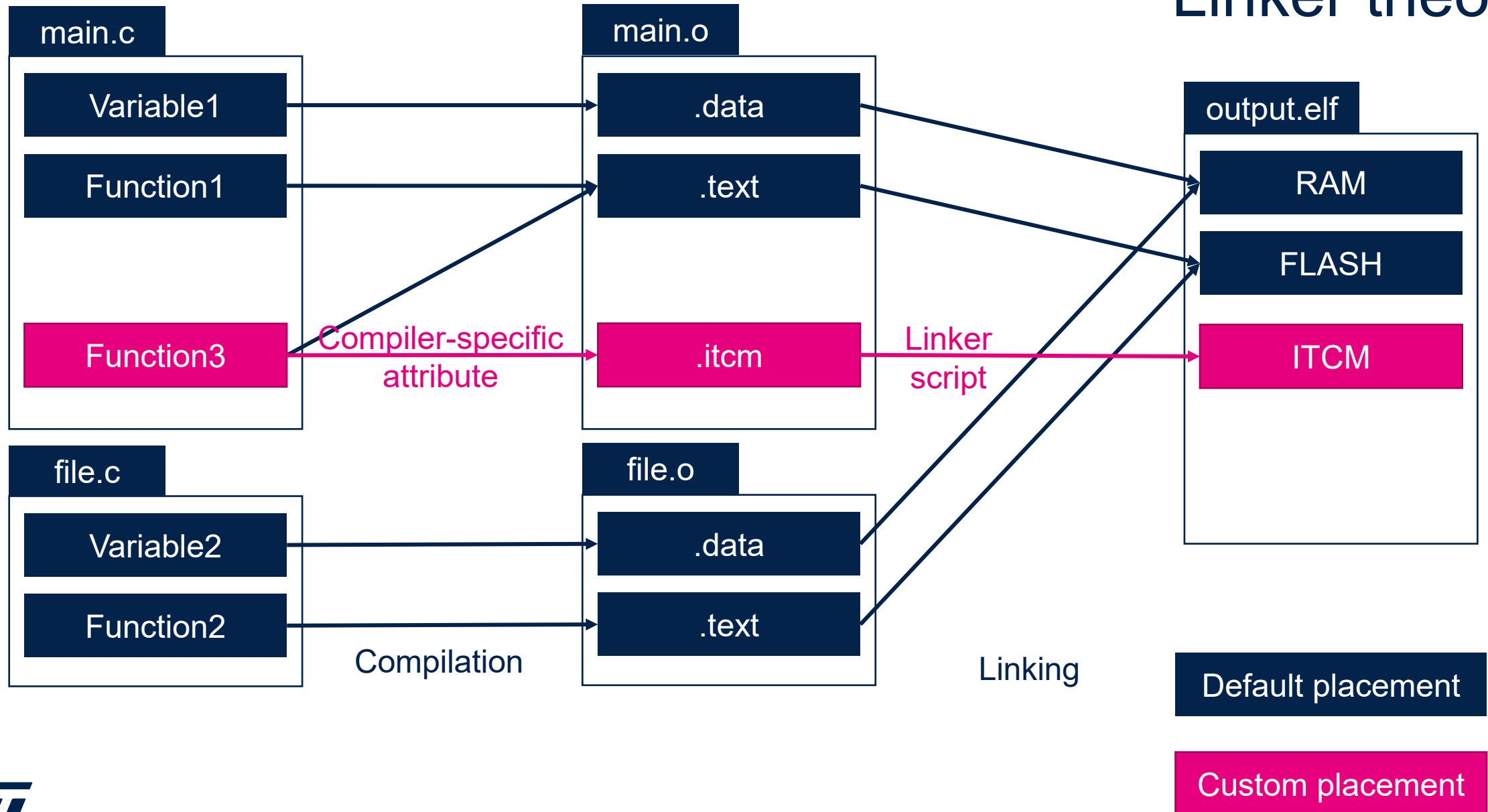
theory



practice



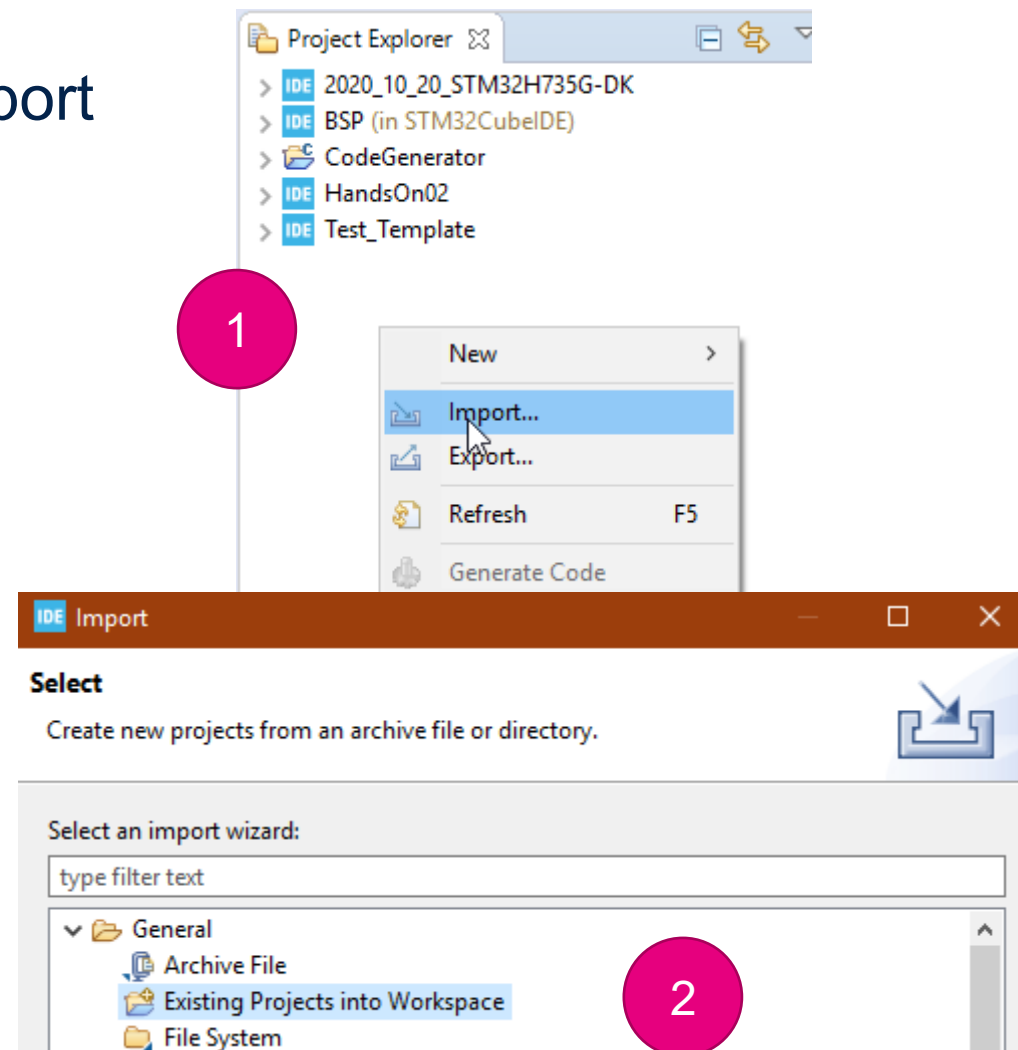
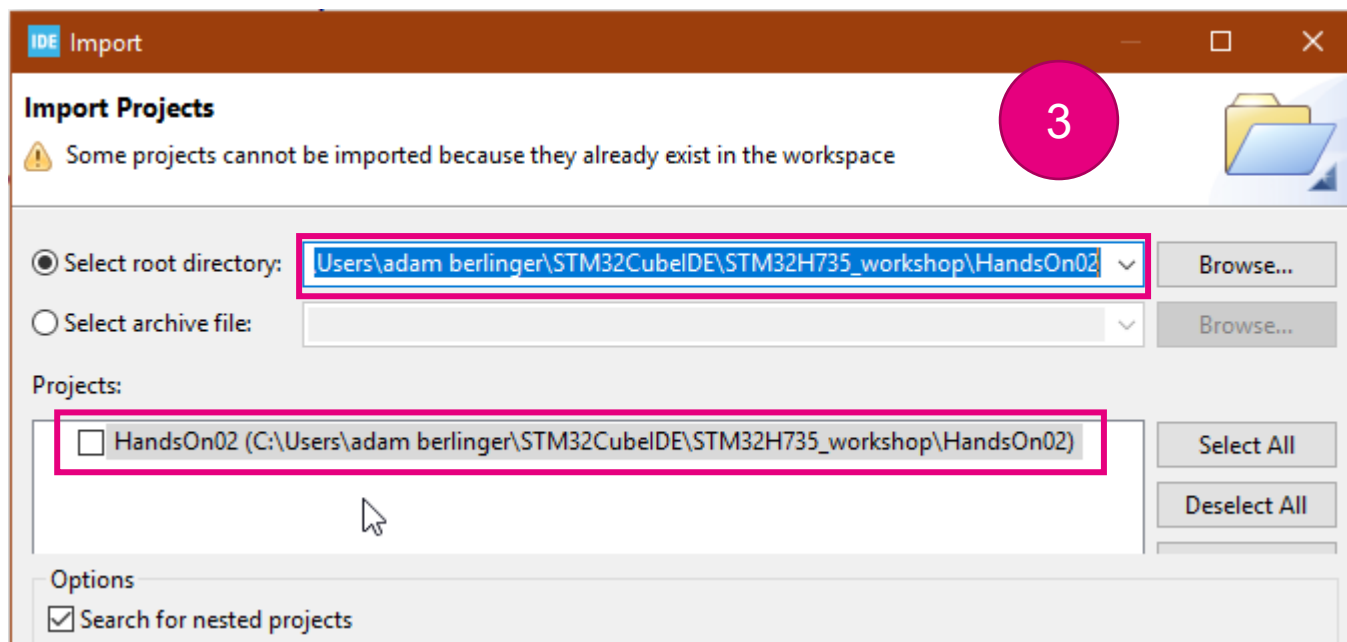
Linker theory





Import the project template

1. Right click in project explorer and select Import
2. Select Existing Projects into Workspace
3. Select path to the template project and select it below

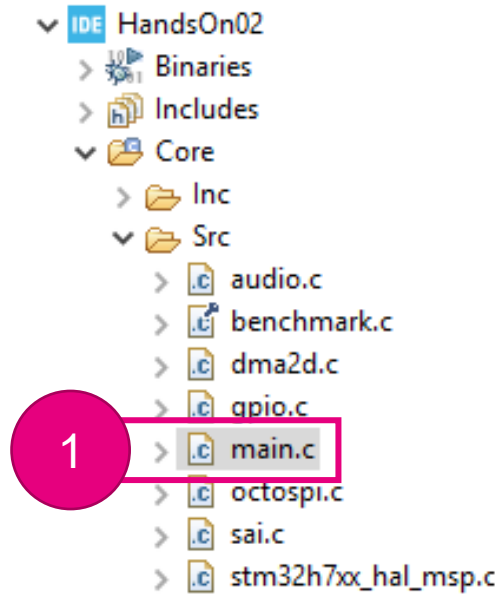




Placing code to separate section

1. Open main.c file
2. Find mainloop_measure function
3. Add following `__attribute__` before function (supported by GCC and Keil):

```
__attribute__((section(".mainloop")))  
void mainloop_measure(void){
```



```
84 /* USER CODE END TV */  
85  
86 /* Private function prototypes -----  
87 void SystemClock_Config(void);  
88 static void MPU_Config(void);  
89 /* USER CODE BEGIN PFP */  
90  
91 /* USER CODE END PFP */  
92  
93 /* Private user code -----  
94 /* USER CODE BEGIN 0 */  
95 __attribute__((section(".mainloop")))  
96 void mainloop_measure(void){  
97     volatile uint32_t var = 1;  
98     uint32_t func_calls = H7Demo.NbFuncCalls;  
99     uint32_t start,end;  
100     uint32_t temp_results = 0;  
101
```



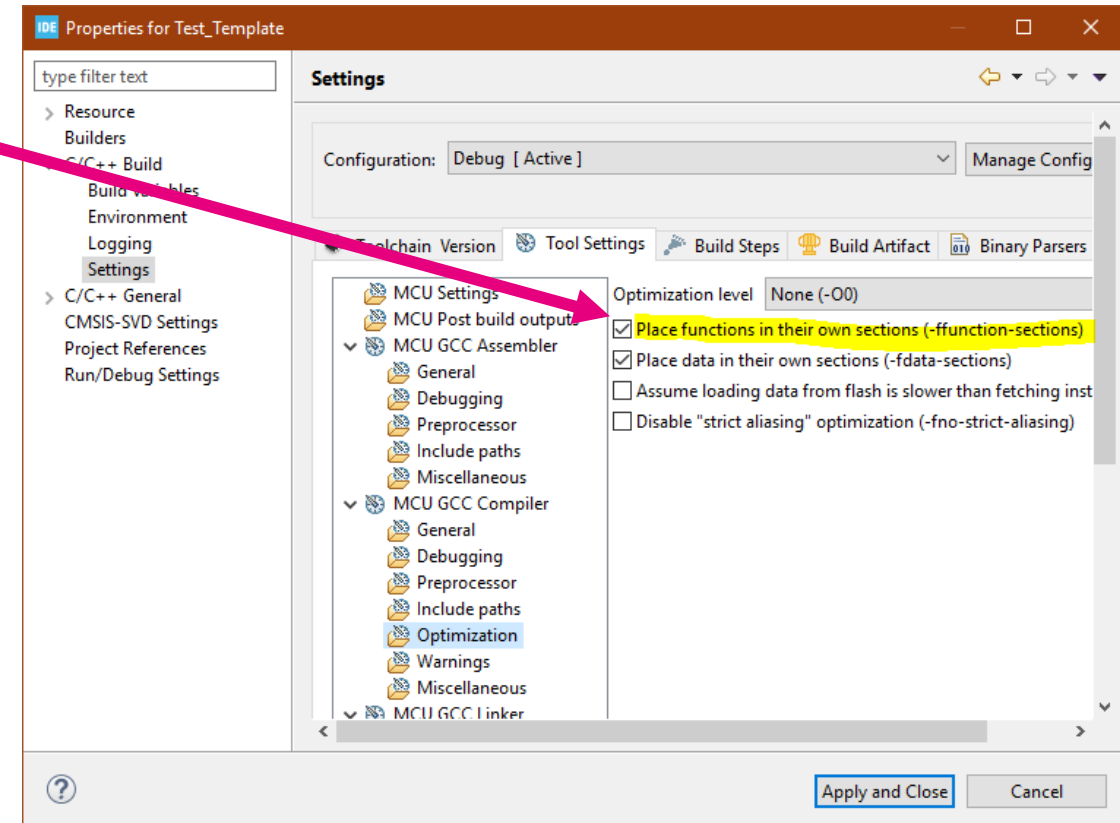
Placing code to separate section

- Using `__attribute__` before function (supported by GCC and Keil):

```
__attribute__((section(".mainloop")))  
void mainloop_measure(void){
```

- Using “-ffunction-sections” GCC option
 - Each function is placed to separate section
 - E.g. main function is placed to “.text.main” section
- IAR
 - `#pragma location=".mainloop"`
- Universal macro for GCC/IAR/Keil

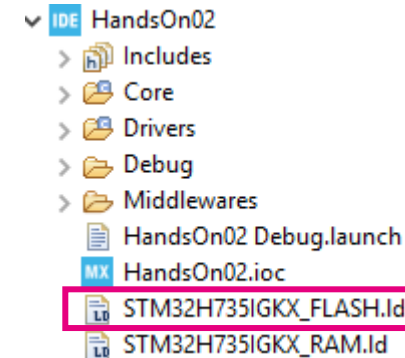
```
#if defined( __ICCARM__ )  
#define MAINLOOP \  
    _Pragma("location=\".mainloop\"")  
#else  
#define MAINLOOP \  
    __attribute__((section(". mainloop")))  
#endif
```





Placing the functions in linkerscript

1. Open file STM32H735IGKX_FLASH.ld
2. Add following code after .isr_vector section
3. Make sure .isr_vector section remains first



```
.isr_vector :  
{  
    . = ALIGN(4);  
    KEEP(*(.isr_vector)) /* Startup code */  
    . = ALIGN(4);  
} >FLASH
```

Load address (flash)

Place address (RAM)

Placing ".mainloop" sections
from all files (* character)

End of section (RAM)

Placing in ITCM RAM

```
/* Used by the startup to initialize data */  
_siitcm = LOADADDR(.itcm);  
.itcm :  
{  
    _sitcm = .;  
    *(.mainloop)  
    _eitcm = .;  
} >ITCMRAM AT> FLASH
```

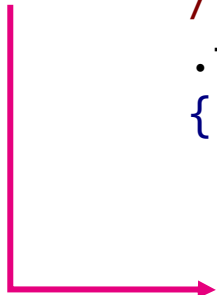
Initialization stored in FLASH



Placing the functions in linkerscript

- Section selection syntax:
 - <file path>(<section name>)
 - '*' acts as wildcard
 - **(.text*)* selects all sections starting with ".text" from all files
 - **benchmark.o(.text)* selects all ".text" section from file paths ending with "benchmark.o"
 - Using '*' is easier than finding out what is the exact path to the file
- When using separate section (.text.mainloop), this should be placed before generic placement

```
/* The program code and other data into "FLASH" Rom type memory */  
.text :  
{  
    . = ALIGN(4);  
    *(.text)  
    *(.text*)  
    ...  
} >FLASH
```





Load data at startup

1. Open main.c file again
2. Add following code in `/* USER CODE BEGIN PV */` use code section:

```
extern uint32_t _siitcm;  
extern uint32_t _sitcm;  
extern uint32_t _eitcm;
```

3. Add following code at beginning of main function (`/* USER CODE BEGIN 1 */`):

```
memcpy(&_sitcm, &_siitcm, (uint32_t)&_eitcm - (uint32_t)&_sitcm);
```



Load data at startup

- For GCC we need to do manually in C:

```
extern uint32_t _siitcm;  
extern uint32_t _sitcm;  
extern uint32_t _eitcm;  
...  
memcpy(&_sitcm, &_siitcm, (uint32_t)&_eitcm - (uint32_t)&_sitcm);
```

- Other option is to use assembly (startup_stm32h735igkx.s)
 - This is already implemented here for “.data” section
 - variables in RAM with non-zero initialize value
 - Might be necessary if all code is in RAM including memcpy function
- IAR and Keil should initialize automatically



Check inside build analyzer

- Information parsed by STM32CubeIDE from MAP file
- We can see if the linkerscript works properly
- We can also observe “veneer” functions
 - Veneer is stub when calling far located functions
 - Branch/jump instructions have limited range
 - Here the mainloop_measure is located ITCM memory, but SCB_InvalidateICache is located in FLASH
 - In some cases we want to avoid calling functions located in FLASH

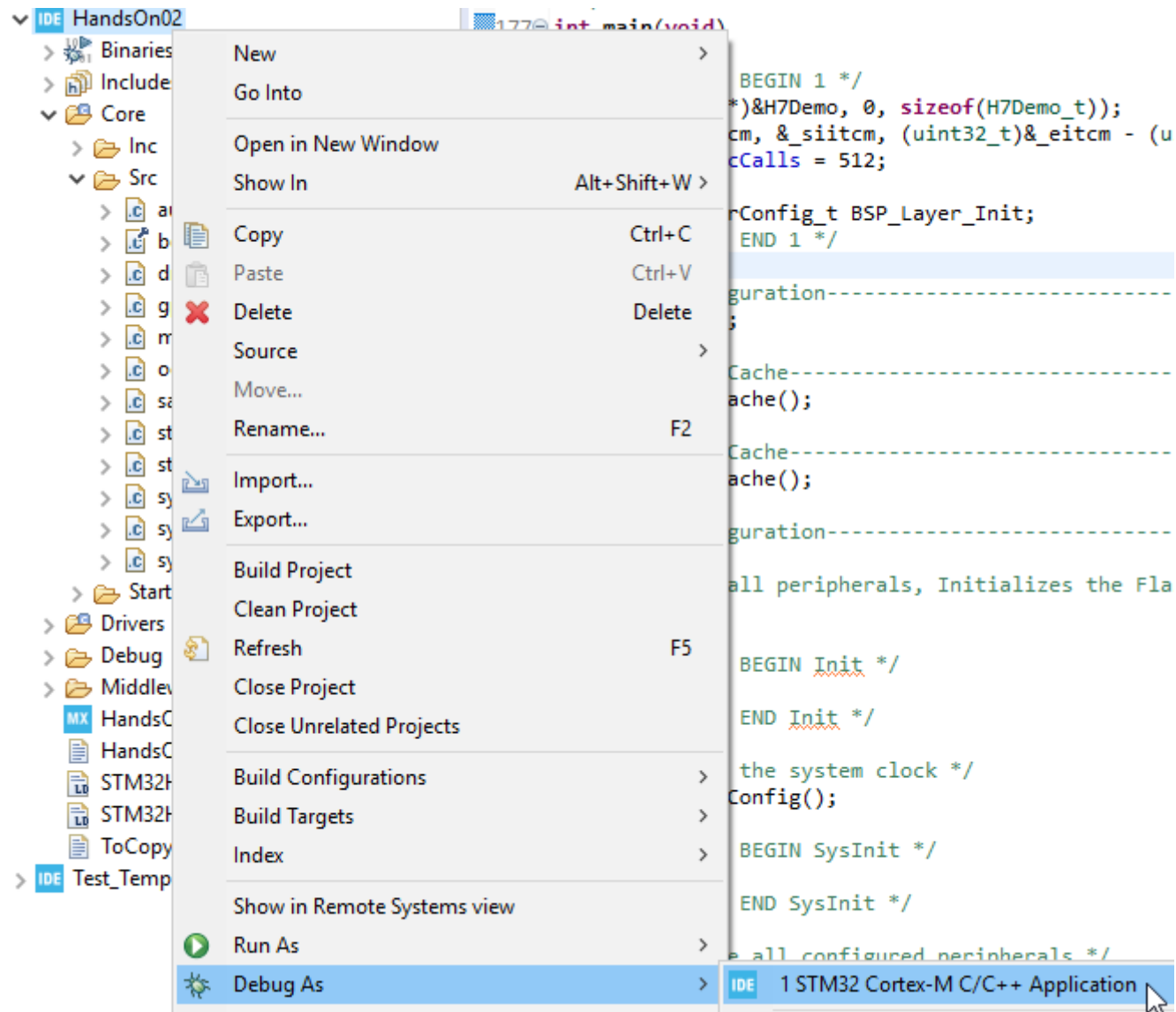
The screenshot shows the 'Build Analyzer' window in STM32CubeIDE. The title bar includes 'Build Analyzer', 'Static Stack Analyzer', and a search icon. The main content area shows the file 'Test_Template.elf' and the path '/Test_Template/Debug' with a timestamp. Below this, there are tabs for 'Memory Regions' and 'Memory Details'. The 'Memory Regions' tab is active, displaying a table of memory regions. The table has columns for Name, Run address (VMA), Load address (LMA), and Size. The regions listed are ITCMRAM (64 KB) and its sub-region .itcm (176 B). Within the .itcm region, two functions are listed: mainloop_measure (168 B) and __SCB_InvalidateICache_veneer (8 B). The __SCB_InvalidateICache_veneer function is highlighted in blue.

Name	Run address (VMA)	Load address (LMA)	Size
ITCMRAM	0x00000000		64 KB
.itcm	0x00000000	0x080002cc	176 B
mainloop_measure	0x00000000	0x080002cc	168 B
__SCB_InvalidateICache_veneer	0x000000a8	0x08000374	8 B



Start debug

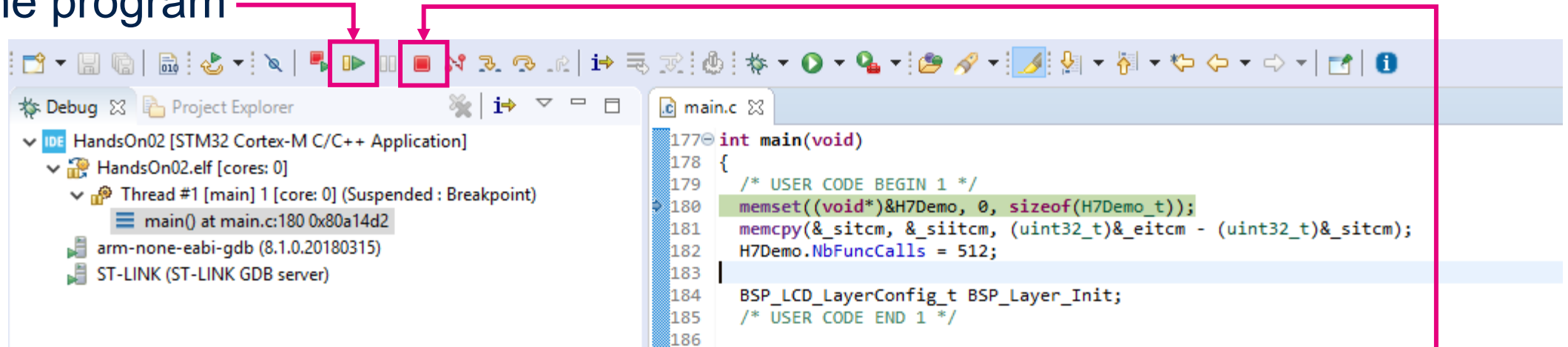
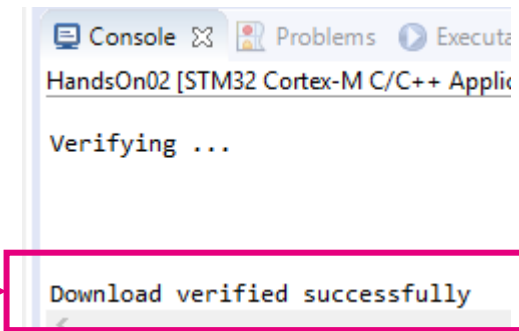
- Make sure the STM32CubeMonitor has stopped acquisition
- Right-click and select “Debug As” > “STM32 Cortex-M C/C++ Application”
- Confirm default debug configuration





Running the program

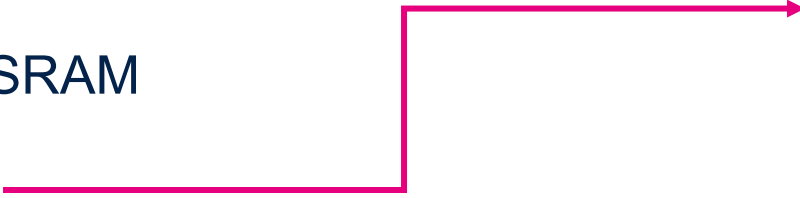
- Wait until the program is loaded
 - This might take some time due to image stored in flash
 - Running the program before it is loaded might cause error
- Run the program

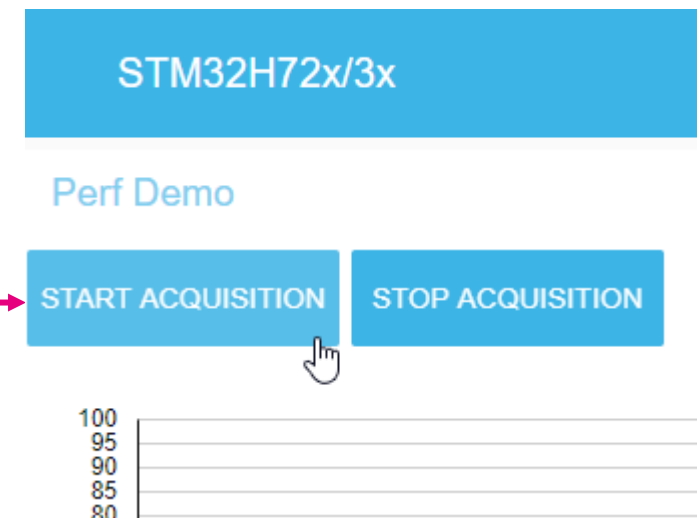


- Close the debug session in STM32CubeIDE
 - So we can connect STM32CubeMonitor



Running STM32CubeMonitor

- Start STM32CubeMonitor (if not already running)
- No need to update the ELF file, since we use same memory layout
 - Variables are at the same part of SRAM
- Start Acquisition in dashboard 
 - Otherwise the toggle switches won't work
- Try different configurations
 - Enabling the Video stream shouldn't cause big performance drop (usually around 1%)
 - Increasing function number above 1024 will “overload” cache





Expected results

Cache ON/OFF	LCD ON/OFF	Functions	Expected performance
Cache OFF	LCD OFF	512	73%
Cache ON	LCD OFF	512	100%
Cache OFF	LCD ON	512	72%
Cache ON	LCD ON	512	100%
Cache OFF	LCD OFF	1080	73%
Cache ON	LCD OFF	1080	81%
Cache OFF	LCD ON	2048	72%
Cache ON	LCD ON	2048	75%



Conclusion

- Internal Flash offers good performance even when ICache disabled
- Additional bandwidth on the internal flash doesn't affect performance significantly
- Similar approach can be used for placing data
- Similar method can be used for your specific application

Thank you

© STMicroelectronics - All rights reserved.

ST logo is a trademark or a registered trademark of STMicroelectronics International NV or its affiliates in the EU and/or other countries.

For additional information about ST trademarks, please refer to www.st.com/trademarks.

All other product or service names are the property of their respective owners.



life.augmented

Using memory protection unit (MPU)

- Disabling cache for DMA buffers
 - Otherwise the DMA or core can read old data
- Prevent speculative access
 - This can cause some issues with uninitialized memories
- Prevent access to certain part of memory
 - More useful for safety than security
 - Beware that the MPU doesn't affect DMA access
- Using MPU on STM32H7 is necessary in most cases
 - MOOC:
<https://www.youtube.com/watch?v=6IUfxSAFhlw&list=PLnMKNibPkDnEQXu4S6QUUHuSKj81MeqCz>
 - AN4838:
https://www.st.com/content/ccc/resource/technical/document/application_note/group0/bc/2d/f7/bd/fb/3f/48/47/DM00272912/files/DM00272912.pdf/jcr:content/translations/en.DM00272912.pdf



Image import

- Using GIMP
- Swap red and blue channels (different order used by LTDC)
 - Can be done in “Color” > “Components” > “Channel mixer”
- Export to C-file
 - There is additional option to export to H file, but this seems to not work
 - Compiler then reports some issues
- Change extension to header file and #include

Project structure

- Template created with CubeMX
- Added BSP from STM32Cube_FW_H7_V1.8.0\Drivers\BSP\STM32H735G-DK
 - Used for driving LCD and Audio
- Some HAL drivers are added manually
 - Drivers\BSP\HAL_dependencies
- benchmark.c/.h files generated by custom generator (written in C)
 - Simple program that generates specific number of functions with same structure
 - Constants are randomized, but there are some restrictions
 - E.g. 0 can be easily optimized out by compiler